



Arquitectura de Software (75.73/TB034)

Trabajo Práctico N°1

1°C 2025

Integrantes del Grupo		
Nombre y apellido	Padrón	Correo
Joaquin Batemarco	110222	jbatemarco@fi.uba.ar
Tomas Emanuel	108026	temanuel@fi.uba.ar
Sofia Javes	107677	sjaves@fi.uba.ar
Theo Miguel	106438	tmiguel@fi.uba.ar

Índice

1. Introducción	3
2. Atributos de calidad	3
2.1. Análisis de QAs en la arquitectura base	3
2.2. QAs Claves	4
2.2.1. Fiabilidad	4
2.2.2. Performance	4
2.2.3. Disponibilidad	5
2.2.4. Escalabilidad	5
2.2.5. Security	6
2.3. QAs no claves	6
2.3.1. Visibilidad	6
2.3.2. Modificabilidad	7
2.3.3. Usabilidad	7
2.3.4. Maintainability	7
2.3.5. Testability	7
3. Caso Base	7
3.1. Arquitectura	7
3.2. Components & Connectors	8
4. Mejoras	8
4.1. Cambios aplicados	8
4.2. Components & Connectors	9
5. Escenarios	10
5.1. Spikes	11
5.1.1. All	14
5.1.2. Negative Balance	17
5.1.3. Per-Currency	20
5.1.4. Escenario 2 - Recovery	23

1. Introducción

El presente trabajo tiene como objetivo analizar, evaluar y mejorar la arquitectura del servicio de cambio de monedas perteneciente a **arVault**, una fintech en etapa de crecimiento que busca garantizar la calidad y confiabilidad de su plataforma antes de avanzar hacia nuevas rondas de inversión.

Para esto, frente a un escenario inicial presentado se realizaron los siguientes análisis:

- Identificación de atributos de calidad (QAs)
- Mediciones y análisis del comportamiento actual del sistema bajo diferentes condiciones de carga
- Análisis de arquitectura del sistema (elaboración de diagrama de Components and Connectors)

2. Atributos de calidad

Los atributos de calidad (QAs) son características no funcionales de un sistema que determinan su éxito en producción. En el contexto del sistema de arVault, estos atributos son fundamentales debido a la naturaleza financiera y el potencial de crecimiento de la plataforma.

2.1. Análisis de QAs en la arquitectura base

Con la arquitectura ya establecida, podemos observar cómo los QAs que determinamos clave se ven afectados por las decisiones de diseño que tomó el desarrollador.

Seguridad: Como la autenticación y la autorización se dejan en manos de VaultSec, podemos darlos por cubiertos. Además de esto, como no se guarda información sensible de los usuarios, la confidencialidad no es una necesidad tan importante. Pero por otro lado, no hay integridad: cualquier usuario puede modificar los balances de las cuentas de ArVault.

Fiabilidad: La app base que se nos disponibiliza tiene fallas intrínsecas como condiciones de carrera que dejan balances negativos, incapacidad de retomar transacciones en caso de caídas de servicio y pérdida los balances de las cuentas de la empresa en caso de corrupción de archivos.

Disponibilidad: En esta arquitectura que recibimos Nginx es un Single Point of Failure (SPF), pero es un riesgo asumido, es decir, se espera que este componente tenga una muy alta disponibilidad, pero, por otro lado, la API expuesta por el desarrollador puede tener tiempos de respuesta elevados en caso de tener un alto tráfico.

Escalabilidad: El uso de Nginx permite una fácil escalabilidad horizontal, pero lamentablemente la API desarrollada no es igual de susceptible a este tipo de mejoras, por lo que se necesitaría primero hacerle cambios antes de poder aprovechar al máximo esta posibilidad que brinda el proxy.

Performance: Con solo analizar el código ya podemos ver posibles mejoras que impactan de manera positiva en este atributo, como puede ser buscar una forma de paralelizar las llamadas a 'transfer()'. Pero por otro lado, el hecho de trabajar en memoria permite tener tiempos de acceso a datos prácticamente nulos.

Visibilidad: La arquitectura ya incluye un sistema de métricas con Graphite, StatsD, Grafana y cAdvisor, lo cual aporta buena capacidad de observación del sistema en tiempo de ejecución, permitiendo visualizar métricas de rendimiento, consumo de recursos y latencias de los servicios. Sin embargo, la API no expone endpoints de health-check ni métricas propias, lo que limita la visibilidad de fallas funcionales (errores en operaciones de exchange o inconsistencias en balances). Cabe mencionar que el sistema de logs desarrollado es un buen acercamiento para un seguimiento de transacciones.

Modificabilidad: La arquitectura modular basada en servicios independientes (API, Nginx, métricas) facilita la modificación o reemplazo de componentes sin afectar al resto del sistema. Sin

embargo, hay varias cosas que revisar. La persistencia basada en archivos JSON limita la escalabilidad y complica los cambios estructurales de datos. Además, la dependencia fuerte del estado en memoria implica que modificar el modelo de datos requiere revisar múltiples puntos del código. Por otro lado, que toda la lógica de negocio se encuentre en un solo servicio monolítico hace que agregar nuevas funcionalidades sea más complejo de lo que debería.

Usabilidad: Desde la perspectiva del cliente (consumidor de la API), la arquitectura provee una interfaz clara vía HTTP/JSON, lo que facilita la integración y el uso por parte de otros sistemas. Sin embargo, no hay documentación formal ni especificación de endpoints (por ejemplo, OpenAPI/Swagger), lo que reduce la facilidad de uso para desarrolladores. Tampoco existen mensajes de error detallados o mecanismos de retroalimentación ante fallas, lo cual afecta la experiencia del usuario al interactuar con la API.

2.2. QAs Claves

Ahora que ya sabemos cuáles son los Atributos de Calidad que tenemos en cuenta y su relación con la arquitectura de la aplicación, veamos cuáles de estos consideramos Clave y qué problemas identificamos en el servicio.

2.2.1. Fiabilidad

La corrección y consistencia de la información financiera es esencial en un sistema que maneja dinero real. Impacta en la confianza del usuario, la reputación y el cumplimiento de regulaciones.

- **Race Conditions:** Las operaciones no son atómicas y pueden ocurrir condiciones de carrera cuando múltiples requests acceden y modifican balances simultáneamente. Esto puede resultar en balances negativos o transacciones perdidas.
- **Sin transacciones ACID:** Si una transferencia falla después de actualizar balances, no hay rollback automático y la compensación manual es frágil.
- **Persistencia asíncrona:** Si el servidor se cae entre escrituras, se pueden perder datos ya que no existe un mecanismo ni garantía de durabilidad.

Ejemplo de problema en código:

```
// No hay atomicidad y puede haber condición de carrera
account.balance -= amount;
// Persistencia asíncrona, riesgo de pérdida de datos
await fs.promises.writeFile(filePath, JSON.stringify(data, null, 2));
```

2.2.2. Performance

A nuestra consideración el atributo de calidad más importante de este trabajo. Dicho atributo afecta directamente a la experiencia del usuario en tiempo real. Afecta en cuanto escala el sistema, cómo compite con otras aplicaciones.

Pudimos identificar varios problemas según la performance:

- **Latencia alta por diseño:** Identificamos que cada exchange hace 2 llamadas a transfer() 400-800ms mínimo, siendo esta una operación bloqueante:

```
async function transfer(fromAccountId, toAccountId, amount) {
  const min = 200;
```

```

    const max = 400;
    return new Promise((resolve) =>
      setTimeout(() => resolve(true), Math.random() * (max - min + 1) + min)
    );
  }
}

```

- **I/O síncrono en archivos:** Se identificó que, sí o sí, al escribir en el archivo se hace una operación bloqueante, que afecta directamente a la performance. Además de esto, la operación *JSON.stringify()*, es costosa según arrays grandes. Por último, no se utiliza un buffer ni batch para escribir; esto afecta a las grandes operaciones.

```

await fs.promises.writeFile(filePath, JSON.stringify(data, null, 2));

```

- **Búsquedas lineales:** Se hacen operaciones lineales según la cantidad de cuentas que existen, teniendo pocas cuentas no sería un problema, pero al escalarlo se convierte en un problema de performance.

```

function findAccountByCurrency(currency) {
  for (let account of accounts) {
    if (account.currency == currency) {
      return account;
    }
  }
  return null;
}

```

2.2.3. Disponibilidad

La disponibilidad es fundamental en un servicio financiero que debe estar operativo 24/7. Cualquier downtime puede causar pérdida de operaciones y desconfianza por parte de los usuarios.

- **Single Point of Failure:** Solo existe una instancia de la API. Si se cae, el servicio queda completamente inoperativo; no hay redundancia ni posibilidad de failover.
- **Sin health checks:** No hay ningún mecanismo automático para detectar caídas ni reiniciar servicios automáticamente. Docker y Nginx no monitorean el estado real de la aplicación.
- **Persistencia frágil:** Si el archivo JSON de persistencia se corrompe, la aplicación no puede arrancar y todos los datos pueden perderse.

```

const raw = await fs.promises.readFile(filePath, "utf8");
return JSON.parse(raw); // Si falla → crash

```

2.2.4. Escalabilidad

La arquitectura necesita soportar el crecimiento en cantidad de usuarios y operaciones a futuro, acompañando así el potencial de crecimiento del negocio.

- **Estado en memoria compartido:** El uso de variables globales hace que escalar horizontalmente (agregar instancias) sea inviable sin perder consistencia, porque cada instancia tendría su propio estado.

```
let accounts;
let rates;
let log;
```

- **Sin estrategia de sharding:** Todos los datos están en un solo proceso; no se puede dividir el sistema por región, moneda u otros criterios de distribución.
- **Log sin límite:** El log se guarda en un array en memoria que crece indefinidamente, con riesgo de memory leak y sin ningún tipo de rotación o archivado.
- **Archivos JSON no escalan:** Cada operación de lectura/escritura implica procesar el archivo completo; no hay índices ni queries eficientes.

2.2.5. Security

Es un atributo de calidad esencial porque el sistema administra dinero y datos personales de los usuarios. Esto lo convierte en un objetivo directo de posibles ataques externos (fraude, robo de credenciales, manipulación de transacciones). Sin una base sólida de seguridad, el sistema no puede generar confianza ni cumplir con regulaciones financieras.

Garantizar la seguridad implica prevenir accesos no autorizados, asegurar la confidencialidad de la información y preservar la integridad de las operaciones financieras.

Dicho de otro modo, la seguridad busca evitar que el sistema sea usado de manera maliciosa o indebida, protegiendo tanto los activos económicos como la información de los usuarios.

- **Validación mínima:** El sistema sólo verifica existencia de parámetros, pero no valida tipo de datos, rangos o integridad semántica.
- **Sin autenticación de endpoints administrativos:** Aunque se menciona "vaultSec", hay riesgos si el control no es suficiente.
- **Falta de integridad:** Cualquier usuario puede modificar los balances de las cuentas de ArVault.

2.3. QAs no claves

Luego también determinamos los siguientes QAs no claves:

2.3.1. Visibilidad

Es un atributo de calidad importante para facilitar el monitoreo y la comprensión del comportamiento del sistema, pero no es crítico en comparación con otros aspectos como seguridad o fiabilidad. La visibilidad permite que los administradores y desarrolladores observen el estado del sistema, identifiquen errores y evalúen métricas de rendimiento mediante logs, paneles o trazas.

Aunque mejora la capacidad de diagnóstico y soporte, una menor visibilidad no compromete directamente la operación ni la seguridad de los fondos de los usuarios.

En resumen, la visibilidad contribuye a la transparencia y mantenibilidad del sistema, pero su impacto es indirecto sobre los objetivos principales de ArVault.

2.3.2. Modificabilidad

Representa la facilidad con la que el sistema puede ser modificado, extendido o corregido sin introducir errores o afectar su estabilidad. Si bien resulta valiosa para reducir costos de mantenimiento y facilitar futuras mejoras, ArVault prioriza la consistencia, seguridad y confiabilidad de las operaciones por encima de la velocidad de cambio.

En sistemas financieros, las modificaciones deben ser cuidadosamente controladas, probadas y auditadas, lo que reduce la necesidad de cambios frecuentes o ágiles.

En síntesis, la modificabilidad aporta flexibilidad técnica a largo plazo, pero no es un factor determinante para los objetivos inmediatos de robustez y seguridad del sistema.

2.3.3. Usabilidad

Hace referencia a la facilidad con la que los usuarios pueden interactuar con el sistema y entender su funcionamiento. Aunque una buena experiencia de usuario es deseable, ArVault se orienta principalmente a la ejecución segura, confiable y eficiente de operaciones financieras, donde la prioridad es la precisión y la integridad de los datos.

Una interfaz más simple o atractiva puede mejorar la adopción y reducir errores humanos, pero no resulta crítica para la funcionalidad central del sistema.

En conclusión, la usabilidad contribuye a la comodidad del usuario, pero no afecta directamente la validez ni la seguridad de las transacciones, por lo que no se considera un QA clave.

2.3.4. Maintainability

La mantenibilidad permite evolucionar y corregir el sistema en el tiempo de manera sencilla.

- Código relativamente limpio pero con *TODOs* y falta de tests automatizados.
- Ausencia de logging estructurado y documentos formales de API más allá del README.

2.3.5. Testability

El nivel de testeabilidad afecta la capacidad para verificar el correcto funcionamiento y hacer regresión de errores.

- La función `transfer()` es una simulación, dificultando cubrir casos reales.
- El estado global y dependencias acopladas hacen difíciles los tests unitarios e integración.
- No hay inyección de dependencias ni mocks configurados.

3. Caso Base

3.1. Arquitectura

La arquitectura del sistema actual o inicial puede definirse de la siguiente manera:

ArVault posee una API monolítica detrás de Nginx como reverse proxy, con persistencia local en archivos JSON y una dependencia externa simulada (un servicio de transferencias con latencia de 200–400 ms). No hay existencia ni uso de DBs externas, no hay caché y no hay balanceo (una sola instancia de API).

3.2. Components & Connectors

A continuacion se detallan los componentes que fueron identificados en este caso base presentado:

- Cliente: Es el consumidor de la API. Comunicacion via HTTP/JSON
- Nginx: Es el punto de entrada, el encargado de terminar las conexiones, hacer el routing a la API y puede añadir headers.
- API Monolitica (Node.js + Express):
 - Controladores HTTP
 - Servicios de Dominio como las reglas de negocio
 - Adaptador de persistencia: mantiene estado de memoria y coordina flush a JSON.
 - Job de flush (en background): cada N segundos serializa y escribe el estado a archivos .json
 - Adaptador HTTP (Cliente) hacia servicios de transferencia (simulacion externa).
 - Logging: logs de acceso (a traves de Nginx) y logs de aplicacion (eventos/errores).
- Sistema de archivos: existencia de un directorio montado en el contenedor de la API para los .json
- Servicio de Transferencia (simulacion externa): Se provee una API REST con latencia 200-400ms. Devolucion de exito/fallo.
- VaultSec: Servicio externo encargado de la seguridad y autenticacion.

A continuacion se presenta el diagrama de Components and Connectors para el caso inicial/base:

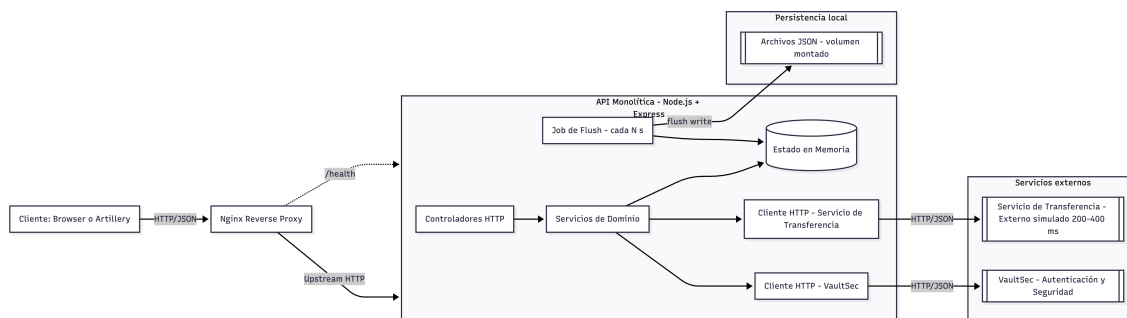


Figura 1: Diagrama de componentes y conectores — arquitectura base.

4. Mejoras

Tras finalizar el análisis implementamos cambios en la arquitectura y el código para tratar de mejorar los atributos de calidad claves.

4.1. Cambios aplicados

Reemplazamos la persistencia de datos en archivos .json locales por una base de datos externa. Esto representa un tradeoff entre performance, disponibilidad y escalabilidad.

La persistencia en archivos locales ofrecía una latencia muy baja y no dependía de servicios externos, lo que favorecía el rendimiento en escenarios simples y reducía puntos de falla. Sin embargo,

este enfoque limitaba la concurrencia, la capacidad de recuperación ante fallas y la integridad de los datos. En cambio, una base de datos externa mejora la consistencia y la tolerancia a fallos, permitiendo múltiples instancias de la API compartiendo el mismo estado y posibilitando la escalabilidad horizontal del sistema. No obstante, introduce mayor latencia y dependencia de red, lo que puede afectar la performance y la disponibilidad en caso de desconexiones.

Por otro lado, modificamos la función de exchange para mejorar la performance y la fiabilidad. Ahora los exchanges se manejan mediante un sistema de commits-rollback, donde primero se hace un hold del monto a extraer que solo se descuenta del balance si se completan las transacciones; en caso de haber algún error en la transacción, se hace la transferencia inversa que si tuvo éxito y se quita el valor del hold.

Este sistema de 'hold' de fondos también cubre el caso de las cuentas con balances negativos, ahora no solo se valida que haya saldo suficiente en la cuenta interna, sino que se necesita que el saldo menos el hold sea suficiente.

Además las llamadas a 'transfer' ahora se hacen de manera concurrente, disminuyendo enormemente el tiempo del exchange.

Agregamos también la protección de modificación del balance de las cuentas o de los rates; ahora estos endpoints requieren de una contraseña para utilizarse, lo que aumenta la prevención de accesos no autorizados.

Luego, una vez hechos todos los cambios, hicimos una reestructuración y refactorización del código, mejorando la mantenibilidad y la modificabilidad. Para esto dejamos un sistema con controladores HTTP, separando responsabilidades en archivos distintos.

En un futuro, se podría migrar a una arquitectura con microservicios, donde cada uno de ellos tenga solo una responsabilidad.

4.2. Components & Connectors

A partir de todos los cambios mencionados presentamos el diagrama de *Components & connectors*.

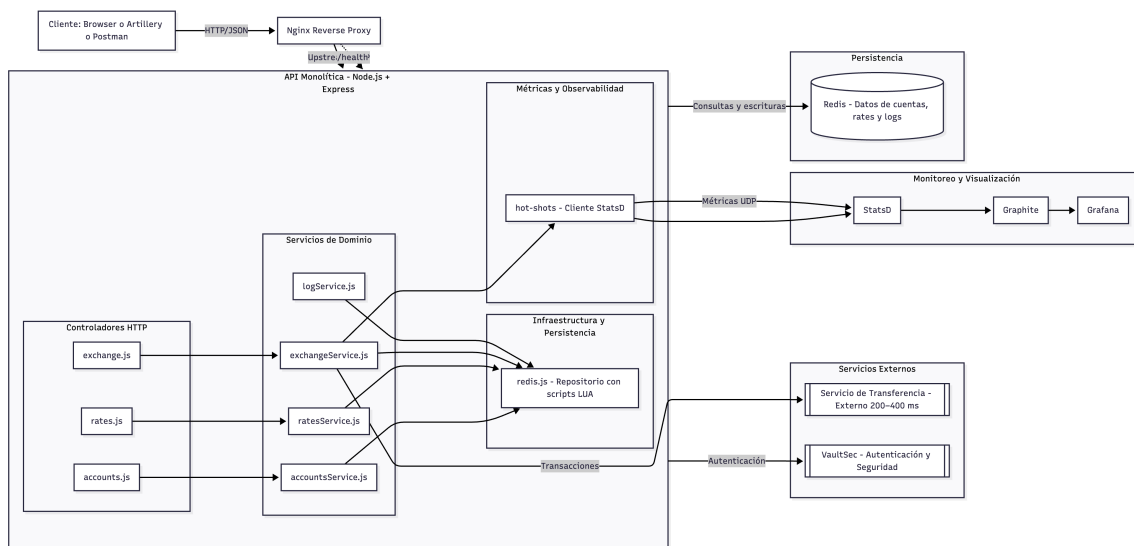


Figura 2: Diagrama de componentes y conectores con los cambios realizados en el sistema

En esta versión mejorada de la arquitectura, el sistema evoluciona desde un monolito con persistencia local hacia una solución más robusta, modular y observable, manteniendo la simplicidad de despliegue bajo una topología monolítica contenida. El objetivo principal de esta etapa fue mejorar la confiabilidad, el rendimiento y la trazabilidad mediante la incorporación de nuevas capas de

infraestructura, métricas y servicios externos.

- **Cliente y capa de entrada:**

- Los usuarios o herramientas de prueba (como Artillery o Postman) interactúan con el sistema a través de Nginx, que cumple el rol de reverse proxy.
- Nginx recibe las peticiones HTTP, aplica reglas de enrutamiento y las redirige hacia la API principal.

- **API Monolítica (Node.js + Express)**

- Organizado en 4 capas:
 - Controladores HTTP: (accounts.js, exchange.js, rates.js) — Gestionan las peticiones entrantes, validan los datos y delegan la lógica a los servicios de dominio.
 - Servicios de dominio: (accountsService.js, exchangeService.js, ratesService.js, logService.js) — Implementan la lógica de negocio y orquestan la comunicación con la capa de persistencia o dependencias externas.
 - Infraestructura y persistencia: (redis.js) — Centraliza las operaciones de almacenamiento, utilizando Redis como base de datos en memoria. Se implementan scripts Lua para garantizar atomicidad en operaciones financieras.
 - Métricas y observabilidad: (hot-shots) — Cliente de StatsD que recolecta métricas de negocio (volumen operado, duración de transacciones, balances) y las envía a Graphite para su posterior visualización en Grafana.

- **Servicios externos**

- Servicio de Transferencia: simula la interacción con un sistema externo de transferencias monetarias, con latencia variable entre 200–400 ms. Se utiliza para evaluar el impacto de dependencias externas en el desempeño.
- VaultSec: servicio de autenticación y seguridad utilizado para validar identidades y operaciones sensibles.

- **Monitoreo y visualización**

- La API envía métricas al agente StatsD, que las reenvía a Graphite, donde se almacenan y agregan.
- Grafana se conecta a Graphite como data source y permite construir dashboards para el seguimiento en tiempo real del sistema.

- **Persistencia**

- Redis reemplaza a los archivos JSON del caso base, lo que permite persistencia en memoria con operaciones atómicas y tiempos de respuesta reducidos.

5. Escenarios

A continuación se detallan los diferentes escenarios que fueron elaborados para observar el funcionamiento y manejo de los componentes de la arVault junto con las diferencias entre el caso base y el caso mejorado.

5.1. Spikes

El objetivo principal de este escenario es medir la respuesta de la API a diferentes niveles de carga, tanto en periodos estables como en picos abruptos de requests.

El escenario consiste en 3 fases principales:

- Warm-up: 30 segundos con una llegada baja (2 req/s) para estabilizar el sistema.
- Spike: incremento inmediato a entre 50 y 80 req/s, con operaciones de cambio entre ARS - USD.
- Cooldown: retorno a una carga baja (3 req/s) para observar la recuperación del sistema.

Caso Base

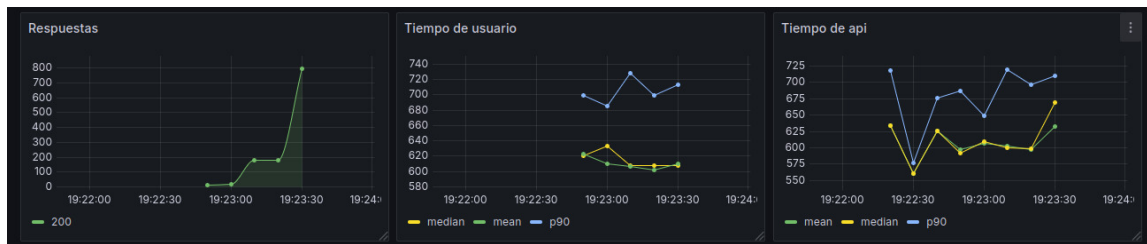


Figura 3: Paneles Scenario Duration y Exchange Duration - Caso Base

En los paneles de *Scenario Duration* y *Exchange Duration* se observó un incremento progresivo de la latencia promedio y del percentil 90 (p90) durante los spikes, alcanzando valores entre 600 y 750 ms. Esto indica que, en alta concurrencia, el sistema experimenta una degradación temporal del tiempo de respuesta, aunque continúa respondiendo correctamente (todas las solicitudes retornaron código 200 OK).



(a) Balance cuenta 1 (ARS)



(b) Balance cuenta 2 (USD)

Figura 4: Comparación de balances de diferentes monedas - Caso Base

Los gráficos de *Account Balances* en este caso evidenciaron un comportamiento coherente y simétrico entre las dos cuentas simuladas:

- La cuenta 1 (verde) incrementa su balance de ARS a medida que vende USD.
- La cuenta 2 (amarillo) reduce su balance de USD en proporción al volumen operado.

Este comportamiento confirma que, incluso bajo condiciones de carga elevada, el sistema mantiene la consistencia de los estados financieros, sin pérdidas ni duplicaciones de información.



Figura 5: Paneles de volumen total acumulado y neto para ambas cuentas

El panel de volumen acumulado refleja visibles escalones en el volumen de USD mientras que el volumen neto operado en ARS demuestra dos visibles spikes negativos de ARS, demostrando que se vende mas ars de lo que se compra.

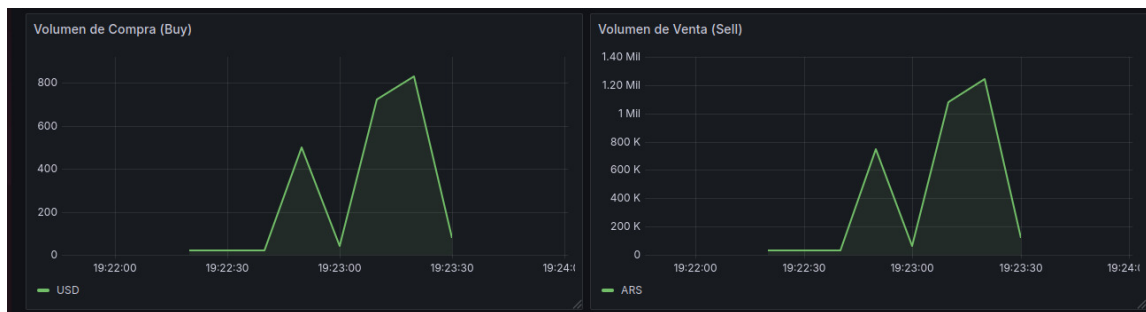


Figura 6: Paneles de volumen de compra-venta en escenario Spike - Caso Base

Los paneles de *Volumen de Compra* (Buy) y *Volumen de Venta* (Sell) reflejan picos sincronizados con las fases del spike:

- Durante los spikes se nota un aumento las ventas de ARS y las compras de USD.

La correspondencia entre ambos volúmenes demuestra que la instrumentación de métricas es correcta y que el registro de operaciones se realiza de manera consistente con el flujo del negocio.

Para este escenario el sistema demostró que logra mantener la estabilidad y la consistencia funcional durante picos de demanda, aunque puede presentar una degradación del tiempo de respuesta al aumentar la concurrencia.

Caso Mejorado

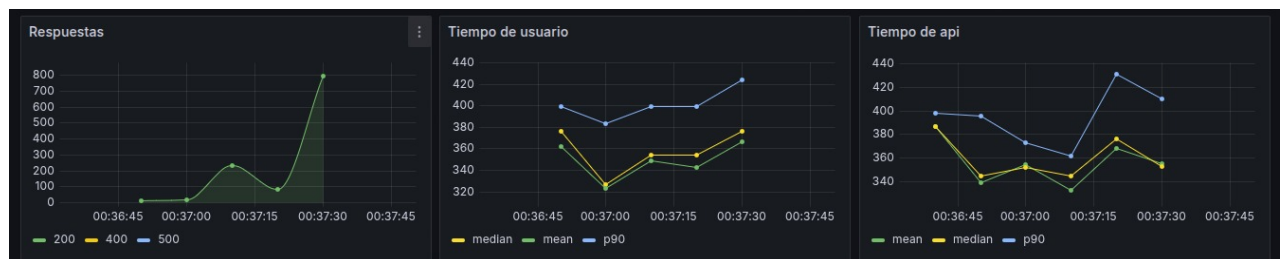


Figura 7: Latencia y códigos durante warm-up, spike, cooldown y spike final - Caso Mejorado

En este caso se observa en el panel de *Tiempo de usuario* que se mantiene en 330–370 ms (mediana) con p90 creciendo a 420–430 ms en el spike final. Luego, en el panel de *Tiempo de API* se observa una oscilación 340–380 ms con p90 cercano a 430 ms en el pico.



Figura 8: Balances internos: drenaje de USD del vault como contraparte del ARS→USD - Caso Mejorado

Para el caso mejorado en el panel de *Balance de cuentas internas* la moneda en USD se drena progresivamente desde 29K hasta 13K durante el spike final lo que es consistente con su rol de contraparte que entrega USD. No hay cruces a saldo negativo.

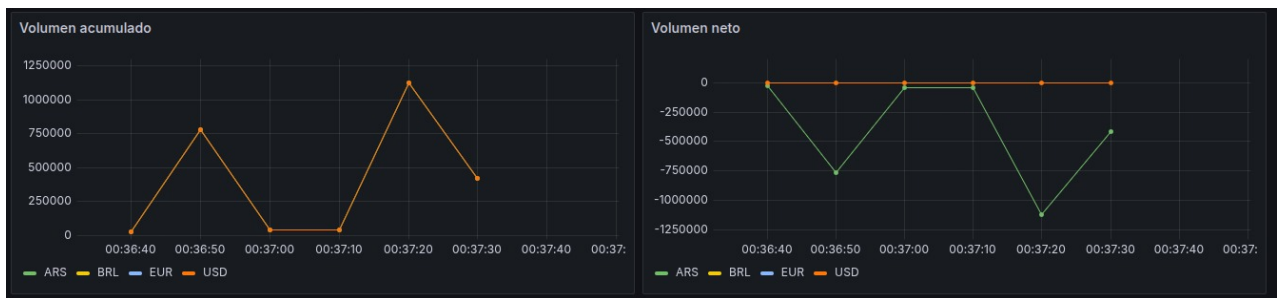


Figura 9: Volumen acumulado (USD) y volumen neto (ARS/USD) - Caso Mejorado

En este caso el panel de *Volumen acumulado* en USD presenta escalones amplios en cada spike y el panel de *Volumen neto* refleja salidas de ARS.



Figura 10: Volumen de Compra (USD) - Caso Mejorado

Los picos de Buy (USD) y Sell (ARS) coinciden con los spikes de carga y muestran simetría temporal entre ellos.

Para este escenario se observa como el sistema es capaz de garantizar que las subidas y bajas de las monedas involucradas sean coherentes entre sí sin operaciones duplicadas a pesar de los altos niveles de requests que se envían.

5.1.1. All

Con el objetivo de analizar el comportamiento del sistema frente a distintos niveles de carga sostenida y picos abruptos, se ejecutó un escenario completo (all) con cinco fases consecutivas:

1. Warm-up: carga baja inicial (2 req/s).
2. Ramp-up: aumento gradual de carga hasta 20 req/s.
3. Soak: carga constante alta (20 req/s durante 3 minutos).
4. Spike: incremento repentino a 100 req/s.
5. Ramp-down: descenso progresivo hasta carga baja nuevamente.

Caso Base

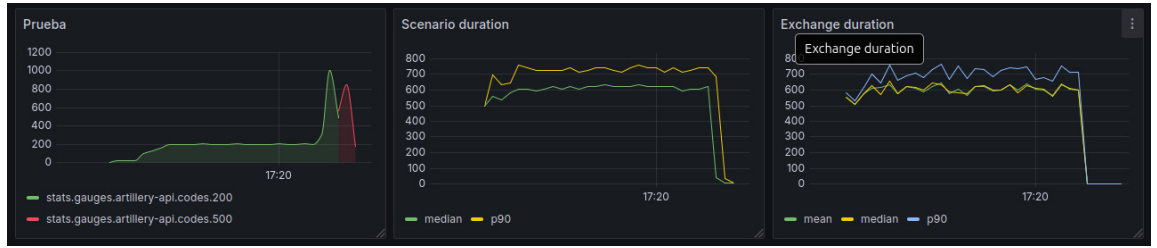


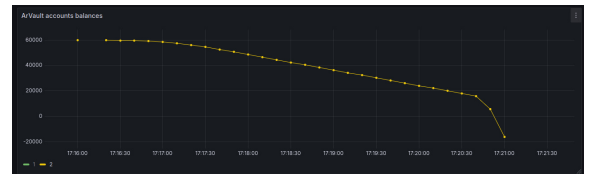
Figura 11: Paneles Scenario Duration y Exchange Duration - Caso Base

Los paneles de *Scenario Duration* y *Exchange Duration* observamos el siguiente patrón:

- Durante el warm-up y ramp-up, la latencia se mantiene estable, con un p95 cercano a 600–650 ms.
- En la fase de soak, el sistema logra sostener esa carga constante sin aumento significativo de latencia, lo cual indica estabilidad bajo carga prolongada.
- En el spike (100 req/s), aparecen los primeros errores HTTP 500 junto con un incremento leve en el p90 (alrededor de 700–750 ms). Esto demuestra que el servidor comienza a saturarse momentáneamente, alcanzando su límite de concurrencia.
- Finalmente, durante el ramp-down, las métricas de latencia y códigos 500 descienden rápidamente, evidenciando una recuperación eficiente tras el pico.



(a) Balance cuenta 1 (ARS)



(b) Balance cuenta 2 (USD)

En este caso se osbervo los siguiente:

- La cuenta 1 (verde) incrementa progresivamente su saldo de ARS, de manera lineal a lo largo del tiempo.

- La cuenta 2 (amarilla) reduce su saldo de USD hasta volverse negativo, reflejando la venta continua de USD durante las operaciones de cambio.



Figura 13: Paneles de volumen total acumulado y neto para ambas monedas.

En el panel de volumen acumulado podemos ver un volumen creciente que luego se sostiene en el rango de los 300k-400k usd. En cambio, el panel de volumen neto denota un volumen negativo de ARS, lo cual coincide con el balance operado de esa moneda contra todas las otras.

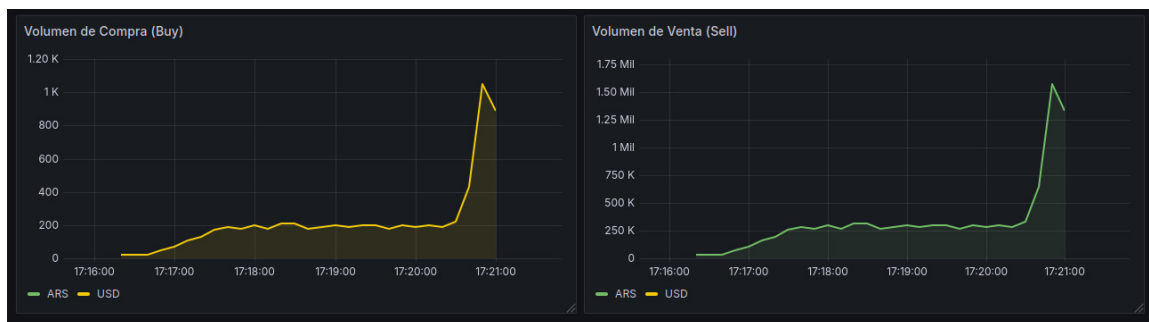


Figura 14: Paneles de volumen de compra-venta en escenario Spike

Los paneles de *Volumen de Compra* y *Volumen de Venta* muestran incrementos graduales durante las fases de ramp-up y soak, seguidos por un pico pronunciado durante el spike (alrededor de 1.2K en USD comprados y 1.5M en ARS vendidos).

Gracias al análisis de este escenario podemos decir que el sistema demostró un comportamiento robusto y predecible durante la mayor parte de la carga, con degradación controlada en el extremo de concurrencia. Esto evidencia que la arquitectura actual (basada en un único servicio monolítico Node.js) es funcional y confiable para volúmenes medios de tráfico, pero no escala linealmente cuando se supera su capacidad de procesamiento concurrente.

Caso Mejorado

A continuación se analizan los cambios del sistema que se reflejan en el escenario que llamamos *All*.



Figura 15: Latencia y códigos - Después

Figura 16: Comparativa de latencias p90/medianas y códigos HTTP - Caso Mejorado

En esta version, las métricas de los paneles de *Tiempo de usuario* y *Tiempo de API* permanecen estables en rangos de 350–420ms (p90 400–420ms) durante warm-up, ramp-up y soak. En la versión base, el p90 del panel *Scenario-Exchange duration* se ubicaba en 700–750ms, con picos de errores HTTP 500 al final del spike.

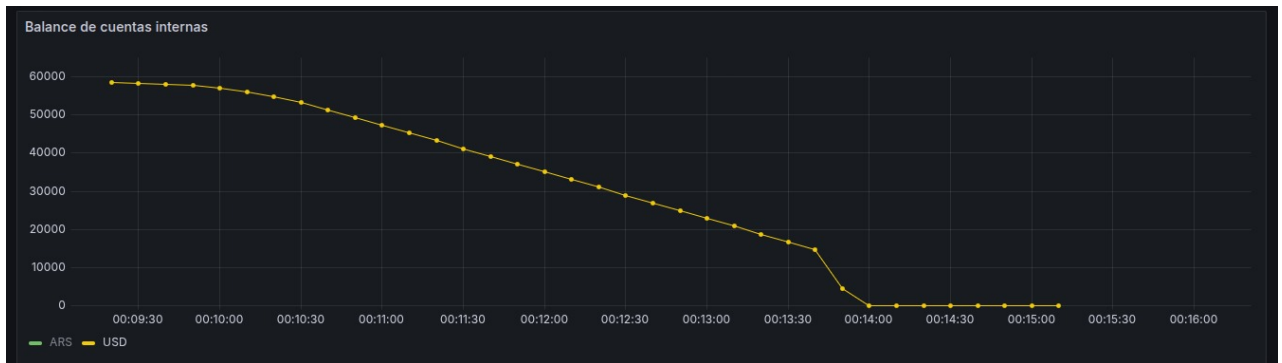


Figura 17: Balances - Caso Mejorado

En la versión mejorada, el balance de cuentas internas desciende hasta 0 y se estabiliza, sin valores negativos, que, a diferencia del caso base que se observan saldos negativos cercanos a -20,000, evidenciando condiciones de carrera que con los nuevos cambios se elimina.



Figura 18: Volumen de compra/venta - Caso Mejorado

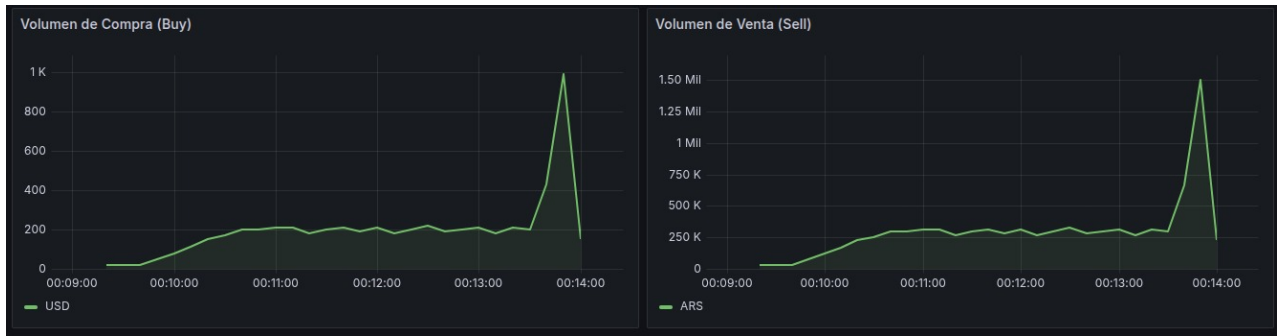


Figura 19: Volumen de compra/venta - Caso Mejorado

En spikes de 100 req/s aún se observa degradación: aparecen ráfagas de 500 y el p90 sube, aunque menos que en la versión base. La arquitectura monolítica limita la escalabilidad horizontal; el cuello de botella persiste en componentes compartidos.

Se podría decir que la versión mejorada se comporta de forma más estable y predecible que la versión inicial. Se redujo la latencia p90 de 700–750ms a 400–420ms, se acotaron los errores 500 a ráfagas breves y se eliminó la posibilidad de balances negativos, lo que evidencia integridad de datos.

La incorporación de persistencia consistente y bloqueos lógicos agrega 30–50ms de overhead base, pero elimina por completo los desbalances negativos y las inconsistencias observadas en la versión base.

5.1.2. Negative Balance

El escenario Negative Balance tuvo como objetivo observar el comportamiento del sistema ante una ráfaga de solicitudes intensas en un tiempo muy reducido, simulando un pico súbito de operaciones.

En este caso el escenario tiene una duración de 10 segundos donde se realizan 50 requests por segundo intentando realizar la conversión de USD a ARS.

Caso base

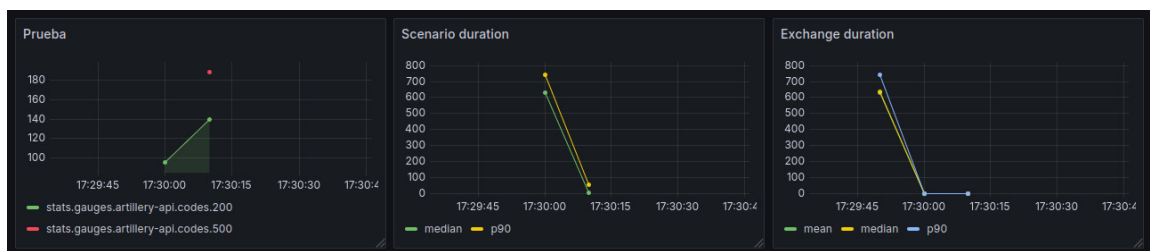


Figura 20: Paneles de Scenario duration y Exchange duration en escenario de Negative Balance - Caso Base

En el caso base paneles de *Scenario duration* y *Exchange duration* reflejan un comportamiento de saturación rápida:

Al inicio del spike, las latencias (mediana y p90) se mantienen dentro del rango de 600–700 ms, son consistentes con los escenarios que analizamos previamente.

Sin embargo, casi de inmediato aparecen respuestas HTTP 500, mientras que el throughput (codes.200) cae bruscamente. Esto indica que el servidor alcanza su capacidad máxima y no logra procesar todas las solicitudes concurrentes dentro del tiempo esperado.

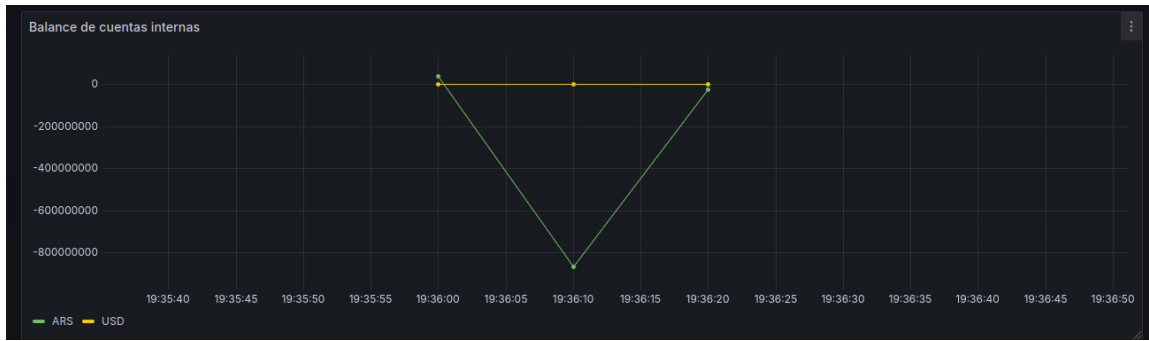


Figura 21: Balances en escenario de Negative Balance - Caso Base

En el panel de account balances para este escenario se observaron variaciones abruptas y signos negativos en los saldos de una de las cuentas. Esto sugiere que:

- Las operaciones de débito y crédito no se aplican de forma estrictamente secuencial, generando un estado intermedio inconsistente.
- Es muy probable que múltiples requests estén intentando modificar el mismo saldo simultáneamente sin bloqueo, ocasionando condiciones de carrera en memoria.



Figura 22: Paneles de Volumen acumulado - neto (USD) - Caso Base



Figura 23: Paneles de Volumen acumulado - neto (ARS) - Caso Base



Figura 24: Paneles de compra-venta operados en escenario de Negative Balance

Los gráficos de *Volumen de Compra* y *Volumen de Venta* mostraron picos pronunciados en el mismo instante coincidiendo entre ellos, seguidos por una caída inmediata a cero. Esto es coherente con un spike breve donde la totalidad de las operaciones ocurre en una sola ráfaga.

Aunque el registro de operaciones demuestra correctitud, la falta de continuidad temporal muestra que la métrica deja de reportar cuando el sistema se satura.

Este escenario fue clave para revelar el punto de quiebre del sistema: la arquitectura actual, que sería el caso base presentado, se basa en un único proceso Node.js con estado en memoria que no maneja correctamente escrituras concurrentes en períodos de alta simultaneidad. El impacto se evidencia tanto en la aparición de errores 500 como en la inconsistencia temporal de los balances.

Caso Mejorado

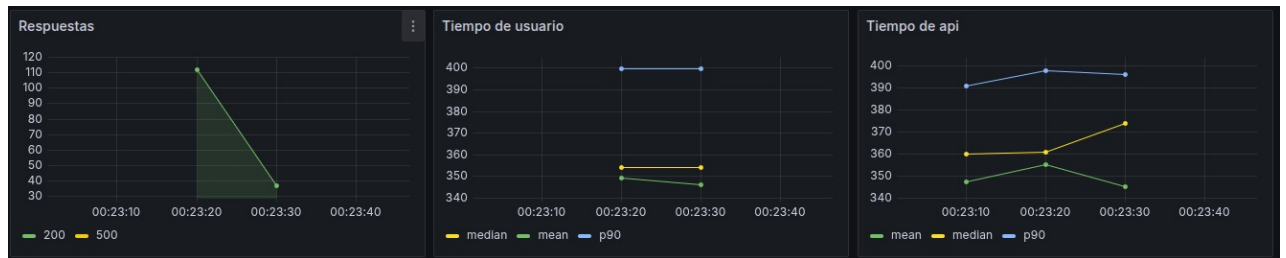


Figura 25: Balances internos — Caso Mejorado

En este caso la latencia se mantiene estable en rangos de 340–400ms (p90 cercano a 400ms) sin explosiones de errores, en cambio, el caso base el p90 del escenario/exchange está alrededor de 700ms.



Figura 26: Volúmenes de compra/venta — Caso Mejorado



Figura 27: Latencia y códigos — Caso Mejorado

En este caso se puede observar como los saldos internos permanecen siempre no negativos y progresan de manera monótona hasta estabilizarse a diferencia del caso base donde puede verse como el saldo cruza cero y cae abruptamente a valores negativos millonarios, reflejando condiciones de carrera y débitos múltiples.



Figura 28: Control de volumen acumulado y neto — Caso Mejorado. El neto acompaña los movimientos sin desbalances.

Los volúmenes de compra/venta en la versión mejorada muestran rampas coherentes y sin picos extremos a diferencia del sistema inicial se observan ráfagas abruptas que coinciden con el desbalance de cuentas. La métrica de volumen neto confirma que el neto se mantiene consistente con los movimientos.

La introducción de estos cambios se podría decir que aumentaron casi minimamente el tiempo de operación pero con el beneficio de que ya no se permiten saldos negativos (la aplicación ahora funciona correctamente).

5.1.3. Per-Currency

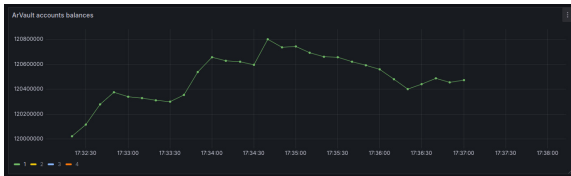
El objetivo de este escenario fue evaluar el comportamiento del sistema ante múltiples tipos de operaciones en diferentes monedas que fueran válidas e inválidas.

Caso Base

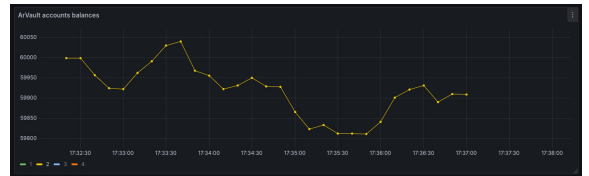


Figura 29: Paneles de Scenario duration y exchange duration en escenario de Per-currency - Caso Base

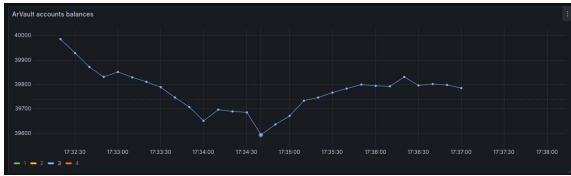
Los gráficos de *Scenario duration* y *Exchange duration* mostraron tiempos promedio estables, con latencias de mediana y p90 entre 600 y 700 ms, lo cual confirma que el rendimiento no se ve afectado por la diversidad de operaciones. Durante las fases con pares inválidos, se registraron respuestas HTTP 500, mientras que las fases válidas mantuvieron un flujo constante de códigos 200.



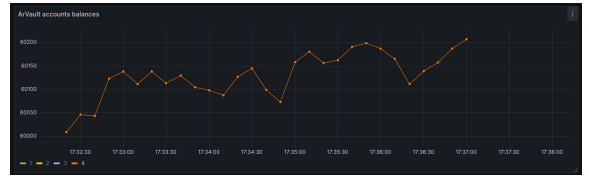
(a) balance cuenta 1



(b) balance cuenta 2



(c) balance cuenta 3



(d) balance cuenta 4

En los paneles de ArVault account balances se observaron una evolución coherente y estable por moneda:

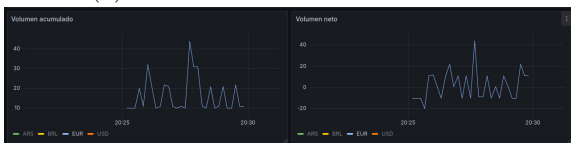
- Cuenta 1 (ARS): mantuvo variaciones de gran escala, lo que reflejo su rol como moneda base.
- Cuentas 2, 3 y 4 (USD, EUR y BRL): presentan fluctuaciones leves y reversibles, coherentes con el tipo y dirección de las operaciones.



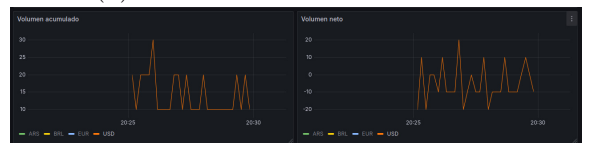
(a) Volumen acumulado-neto ARS



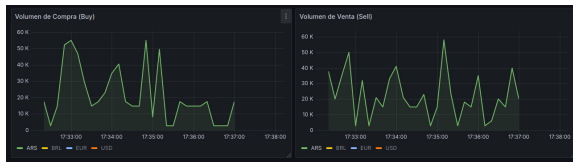
(b) Volumen acumulado-neto BRL



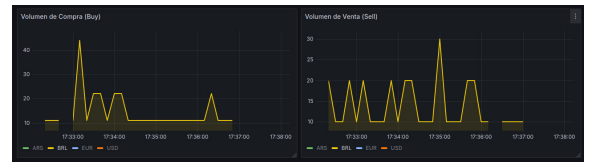
(c) Volumen acumulado-neto EUR



(d) Volumen acumulado-neto USD



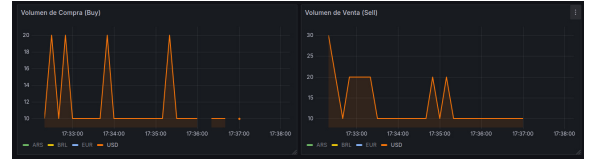
(a) compra-venta ARS



(b) compra-venta BRL



(c) compra-venta EUR



(d) compra-venta USD

Los paneles de Volúmenes (Acumulado y Neto) junto de Compra y Volumen de Venta reflejaron correctamente la segmentación por moneda:

- Cada color (ARS, USD, BRL, EUR) tiene picos sincronizados con las fases correspondientes.
- Las fases inválidas no aportaron volumen, validando que el sistema no registre operaciones rechazadas.

Caso Mejorado

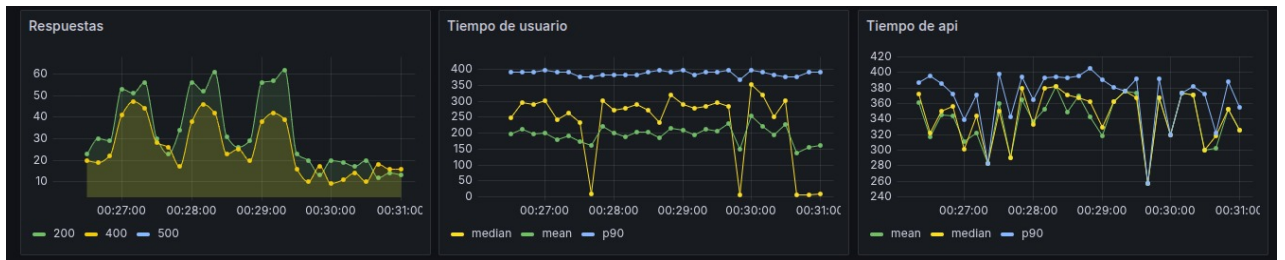


Figura 33: Paneles Tiempo de usuario y Tiempo de api - Caso Mejorado

En este caso se reflejan los nuevos códigos de respuestas 4xx que son consistentes y no impactan en los balances.

En los paneles de *Tiempo de usuario* se observa una mediana de 250–320ms con p90 370–390ms junto con el panel de *tiempo de API* con 320–400ms sin colas sostenidas.

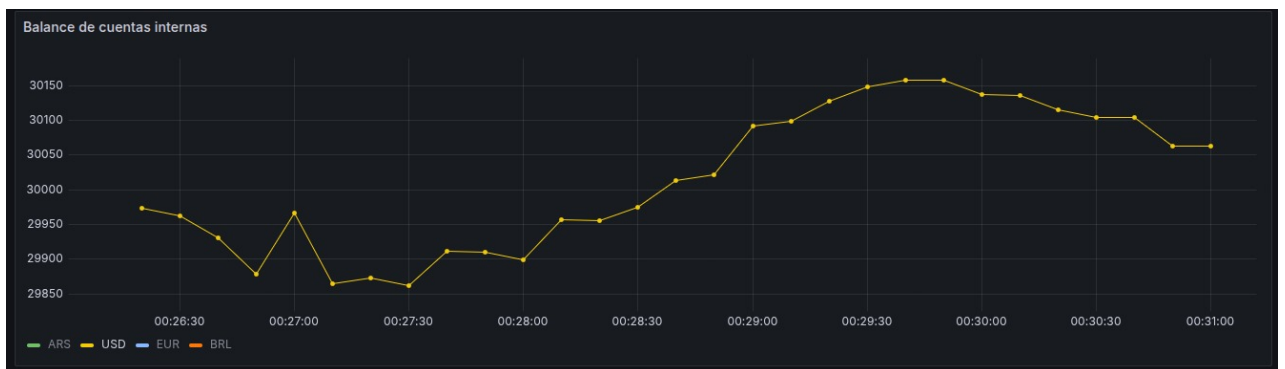


Figura 34: Panel Balance de cuentas internas - Caso Mejorado

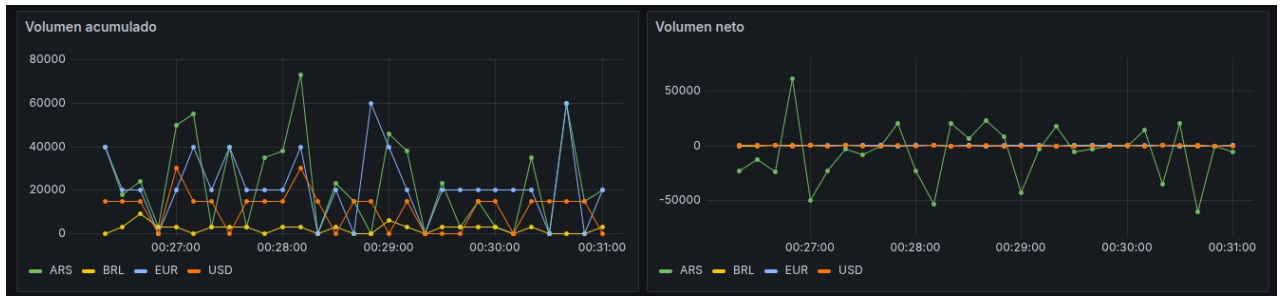


Figura 35: Volumen acumulado y neto por moneda — versión mejorada.

El volumen acumulado es coherente con las fases del escenario y el volumen neto permanece cercano a cero fuera de la moneda activa.

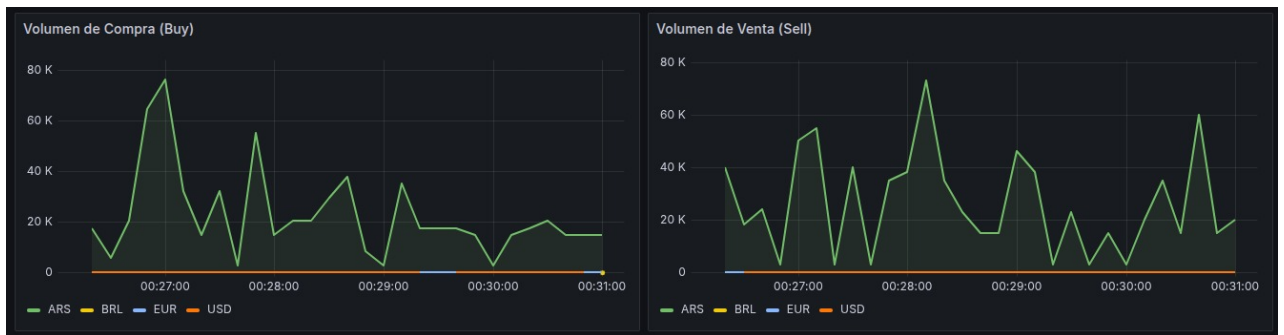


Figura 36: Volumen de compra/venta — versión mejorada.

Los paneles de Buy/Sell de ARS, BRL, EUR y USD en este caso reflejan la alternancia de cargas y no hay actividad errónea por pares no soportados.

El escenario multi-moneda mejorado mantiene latencias estables, rechaza pares inválidos con 4xx sin afectar saldos y evita desbalances. El leve overhead por controles de integridad se compensa con una operación más robusta y predecible.

5.1.4. Escenario 2 - Recovery

Este escenario tiene como objetivo de evaluar la capacidad del sistema para recuperarse después de una falla simulada. El test consiste en tres fases:

1. Warm-up (3 req/s durante 30s)
2. Simulación de caída (10 req/s durante 30s)
3. Recuperación (15 req/s durante 40s)

Caso Base

En el caso base, cuando la API se detenía, no existía un mecanismo automático de recuperación ni una forma de reiniciar el servicio de manera controlada. La dependencia directa del estado en memoria y la falta de health checks implicaba la pérdida definitiva de la información que no se llegó a guardar en memoria al momento previo a la caída. Sin embargo, en los gráficos se podrá observar que las estadísticas retoman con datos similares a los previos a la caída. Esto se debe al guardado periódico en disco (y en volúmenes) del almacenamiento del caso base.



Figura 37: Paneles escenario Recovery - Caso Base

En esta figura podemos observar claramente el corte de los datos en los 3 gráficos y cómo el sistema retoma su funcionamiento normalmente.

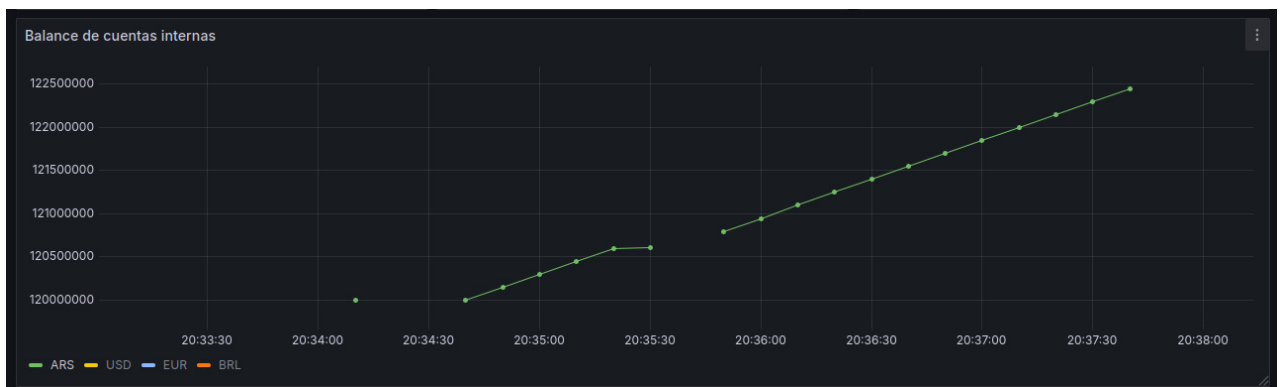


Figura 38: Panel Balance Pesos - Caso Base

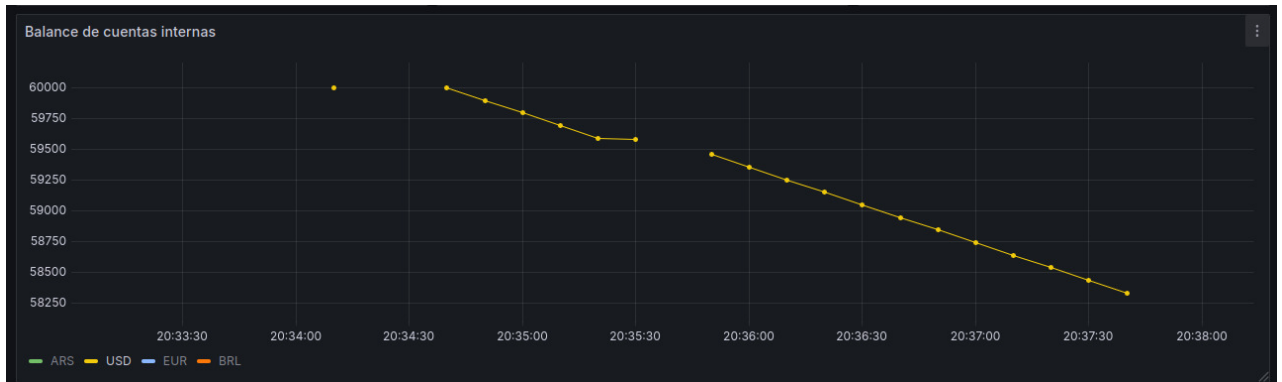


Figura 39: Panel Balance USD - Caso Base

En los paneles de balance de las cuentas de pesos y dólares se observa el comportamiento de ambas de forma inversa, junto con el corte debido a la caída.

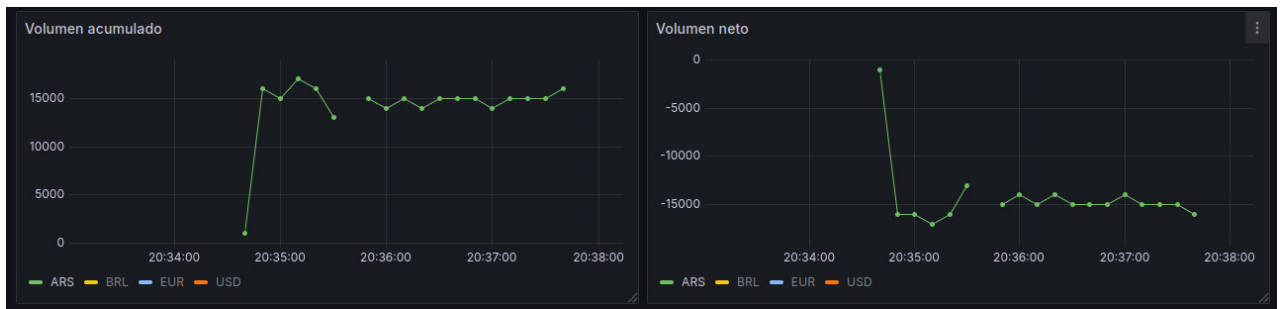


Figura 40: Panel volumen total acumulado y neto en pesos - Caso Base

En estos paneles se puede observar el comportamiento inverso del volumen acumulado vs el neto en pesos. El acumulado sube hasta el rango de los 15000 donde se mantiene, mientras que el neto pasa a valores negativos ya que se venden pesos por dólares.

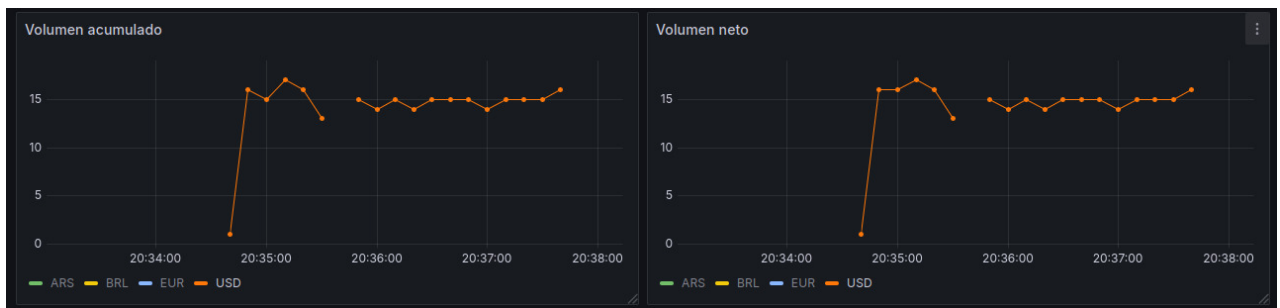


Figura 41: Panel volumen total acumulado y neto en dolares - Caso Base

En estos paneles vemos el comportamiento inverso a los mismos paneles para la moneda ARS, también con el visible corte en la operación del sistema.

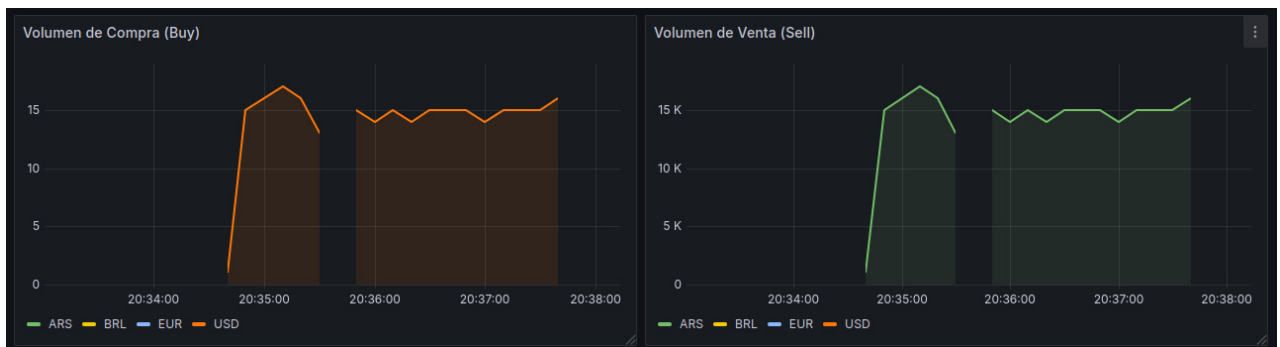


Figura 42: Panel volumen de compra y venta para ambas monedas - Caso Base

En este último panel podemos observar el volumen de compra de USD y de venta de ARS, que siguen el mismo comportamiento. Además puede verse como los datos cortan al momento de la caída simulada y retoman desde donde habían dejado.

Caso Mejorado



Figura 43: Paneles escenario Recovery - Caso Mejorado



Figura 44: Paneles escenario Recovery - Caso Mejorado

Durante la fase de falla, el sistema mostró una caída completa del throughput, con latencias que se dispararon a valores superiores a 2000ms y una tasa de errores del 100 %.

En la fase de recuperación, el sistema tardó aproximadamente 15-20 segundos en estabilizarse completamente. Inicialmente se observaron latencias elevadas (800-1200ms) que gradualmente descendieron a los valores normales de 600-700ms. El throughput se recuperó progresivamente hasta

alcanzar la tasa objetivo de 15 req/s.

La implementación de la base de datos externa mejoró significativamente la consistencia de datos durante la recuperación, evitando pérdidas de información. Sin embargo, introdujo una latencia adicional de 50-100ms en condiciones normales debido a las consultas a la base de datos.